Vapor Stone
RAIL SYSTEMS
A Wabtec company

10 655 Henri-Bourassa Blvd., West
Saint-Laurent (Québec) Canada
H4S 1A1

Tel: 514.335.4200
Fax: 514.335.4231

www.wabtec.com

Vapor Stone Rail Systems

Generic Software Documentation Project

Project # 7260

Software Design Description (SDD)
PTA Protocol

Contract Number		CDRL Number	
		Customer Ref. Number	

NOTICE
Plans, specifications and engineering data submitted herewith or which may be submitted, remain the property of **Vapor Stone Rail Systems** and shall not be reproduced or altered or used in whole or in part except by express consent of **VSRS**.

AVIS
Les plans, spécifications et données techniques soumis ci-joints, ou qui peuvent être soumis, restent la propriété de **Vapor Stone Rail Systems** et ne doivent pas être reproduits, altérés ou utilisés, totalement ou partiellement, sans le consentement express de **VSRS**.

	Name / Title	Signature / Date
Prepared :	Thomas Marmeth Software Designer	2011-10-03
Verified :	Eric Malo, Eng. System Engineer	2011-10-04
Approved :	Martin Ouellet, Eng. Software Department Supervisor	2011-10-04

Document No.	**201111586**	Revision	**1**
		Date (yyyy-mm-dd)	**2011-10-03**

Revision List

Rev.	ECD#	Date	By	Description of changes
1	---	2011-10-03	Thomas Marmeth	Released version.
0.1	---	2011-10-02	Thomas Marmeth	Implemented comments from inspection I-1810
0	---	2011-08-22	Thomas Marmeth	Original issue

Preface

This Software Design Description (SDD) complies with the requirements of the IEEE 1016-1998 and IEEE Std 1558-2005, IEEE Recommended Practice for Software Documentation for Rail Application and Systems approved on 2005-05-23.

Table of Contents

Revision List 2
Preface 3
Table of Contents 4
List of Figures 5
List of Tables 5

1 INTRODUCTION 6

1.1.1 Purpose 6
1.1.2 Scope 6
1.1.3 Definitions and Acronyms 6
1.1.4 Document Organization 8

2 REFERENCES 9

3 SOFTWARE DESIGN 10

3.1 Decomposition description 10
3.1.1 PTA protocol Module Description 10
3.1.2 Concurrent Process Decomposition 10
3.1.3 Data Decomposition 10
3.2 Dependency Description 11
3.2.1 Inter-Module Dependencies 11
3.2.2 Inter-Process Dependencies 12
3.2.3 Data Dependencies 12
3.3 Interface Description 13
3.3.1 PTA Protocol Module functions 13
3.4 Detailed Design 19
3.4.1 Portable Test Application Module Design 19
3.4.2 Data Entities Detailed Design 27

List of Figures

Figure 3-1: Inter-Module Dependencies Diagram 11
Figure 3-2: Inter-Process Dependencies Diagram 12
Figure 3-2: PTA File Request and File Stat Flowchart 21
Figure 3-3: PTE_LoginReq 23
Figure 3-4: PTA Command Interpreter Flowchart 26

List of Tables

Table 3-1: PTA File Type Implemented in DCU Application 20
Table 3-2: Status Code for PTA Login and PTA Logout command 22
Table 3-3: PTA Messages implemented in DCU Application 24

1 INTRODUCTION

This document is the Software Design Description (SDD) of the Portable Test Application (PTA) Protocol Module developed by Vapor Stone Rail Systems.

The PTA Protocol Module can be part of any of the Door Module System (DMS) Applications.

1.1.1 Purpose

The purpose of this Software Design Description is to describe the software components and implementation details of the PTA Protocol Module.

The intended audience is the Vapor Stone Rail Systems software development team and maintenance team members of the transit authority and the car builder.

1.1.2 Scope

The PTA Protocol Module is a software module of the Door Module System (DMS) Application.

1.1.3 Definitions and Acronyms

1.1.3.1 Definitions

Boot Loader	Software starting the target application.
Car Network	Data bus connecting the module with the rest of the train.
Communication Gateway Module	A specific processor based electronic module running a Communication Gateway Module Application. It acts as gateway between the Door Control Module Application and the Car Network/Portable Test Application.
Door Control Module	A specific processor based electronic module running a Door Control Module Application. It is used for controlling the door motor.
Door Control Unit	The final unit that is used to control one or more door panels. This unit is normally made of one or two door control modules running the Door Control Module Application and possibly one Communication Gateway Module Application.
Door Module System	A term that is used for generically identifying a processor based electronic module running any software within any Door Control unit.
HW_COM	Communication interface used for communication between Portable Test Application and Door Module System application, that can be USB, serial or Ethernet.

1.1.3.2 Event Level Definitions

Event:	Term used to globally identify an Info, an Exception or a Failure.
Info:	Info is an occurrence of an Informative condition monitored by the system, which is neither an exception nor a failure.
Exception:	An exception is an occurrence of a condition that is outside of the specified parameters and affecting an outside Line Replaceable Unit (LRU).
Failure:	A failure is an occurrence of a condition that is outside of the specified parameters and affecting this Line Replaceable Unit. A failure normally requires maintenance or replacement to restore affected unit to its normal operation conditions.

1.1.3.3 Acronyms

ASCII	American Standard Code for Information Interchange
CDRL	Contract Data Requirements List
CPU	Central Processing Unit
CRC	Cyclic Redundancy Check
DCM	Door Control Module
DCRP	Door Control Relay Panel
DCU	Door Control Unit
DMS	Door Module System
DOP	Door Operator
ECD	Engineering Change Document
EDF	Event Description File
FLOG	Fault logger
FT	File Transfer
HW	Hardware
ICD	Interface Control Document
IEEE	Institute of Electrical and Electronics Engineers
IRAM	Internal Random Access Memory
KB	Kilobytes
LED	Light Emitting Diode
LH	Left Handle
LRU	Line Replaceable Unit
MDC	Master Door Controller
MHz	MegaHertz
ms	millisecond
NVRAM	Non-Volatile Random Access Memory
OPDF	Operational Parameters Description File
OS	Operating System
PID	Proppotional Integral Derivative
PTA	Portable Test Application
RAM	Random Access Memory
RH	Right Handle
RS232	Recommended Standard 232 (Computer Serial Interface)
RTC	Real-Time Clock
RTDM	Real-Time Data Monitoring
RTS	Request To Send
SCI	Software Configuration Item
SDD	Software Design Description
SRAM	Static Random Access Memory
SRS	Software Requirements Specifications
Std	Standard
T/F	True/False
TS	Technical Specification
USB	Universal Serial Bus
VSRS	Vapor Stone Rail Systems

1.1.4 Document Organization

The Software Design Description (SDD) document describes the architectural software design of the PTA Protocol Module. This SDD is divided into 3 sections and the following specific information is provided within each one:

- Section 1, Introduction, states the purpose, identifies the scope, gives definitions, abbreviations and an overview of the SDD.
- Section 2, References, lists the factors that affect the product and its requirements.
- Section 3, Software Design summarizes followings:
    - Decomposition: describes software entities.
    - Dependency Description: gives a brief description of the dependency between each entity.
    - Interface Description: describes the interfaces of each available function.
    - Detailed Design: details the algorithms of each function implemented for each software entity, presents and explains tasks performed by each software entity.

Within the document, diagrams, flowcharts and tables are used to present the PTA Protocol Module software design:

- Flowcharts give a graphical view of how entities and functions process work.
- Figure 3-1 describes the dependencies between different modules.
- Tables contain data structures used by the PTA Protocol Module. They also present C code.

2 REFERENCES

This section lists the number, title, revision, date, and source of all documents referenced in this SDD.

Standards

IEEE Std 1016-1998	IEEE Recommended Practice for Software Design Descriptions, September 23rd 1998, IEEE, www.ieee.org
IEEE Std 1558-2005	IEEE Recommended Practice for Software Documentation for Rail Application and Systems, March 23rd 2005, IEEE, www.ieee.org

Vapor Stone Rail Systems Documents

S17-07	Software Design Description Template, Rev. 0, 2002-01-15 complemented with requirements from IEEE Std 1558.
CMM-01	Software Engineering Process, Rev.3
CMMI-02	Software Engineering Policy Manual, Rev. 1
201111489	Portable Test Interface Protocol, Interface Control Document (ICD), VSRS, Rev. 01

3 SOFTWARE DESIGN

3.1 Decomposition description

3.1.1 PTA protocol Module Description

3.1.1.1 PTA Protocol Module

The Portable Test Application (PTA) Module is a single entity which receives commands from the Serial Port Driver regarding self-test, snapshot, real-time data monitoring, fault log, and download of a new software. The PTA module sends the results of its command to the Serial Port Driver.

3.1.2 Concurrent Process Decomposition

3.1.2.1 PTA protocol Tasks Description

Two tasks are used by the PTA Module: PTE_TaskCommandInterpreter and PTE_TaskEvent.

PTE_TaskCommandInterpreter is used to receive and execute messages from the Portable Test Application. PTE_TaskEvent is used to monitor and send faults to the connected master end if this one is connected and requests it.

The module functions shall be designed to support multi-threading.

3.1.3 Data Decomposition

Not applicable.

3.2 Dependency Description

3.2.1 Inter-Module Dependencies

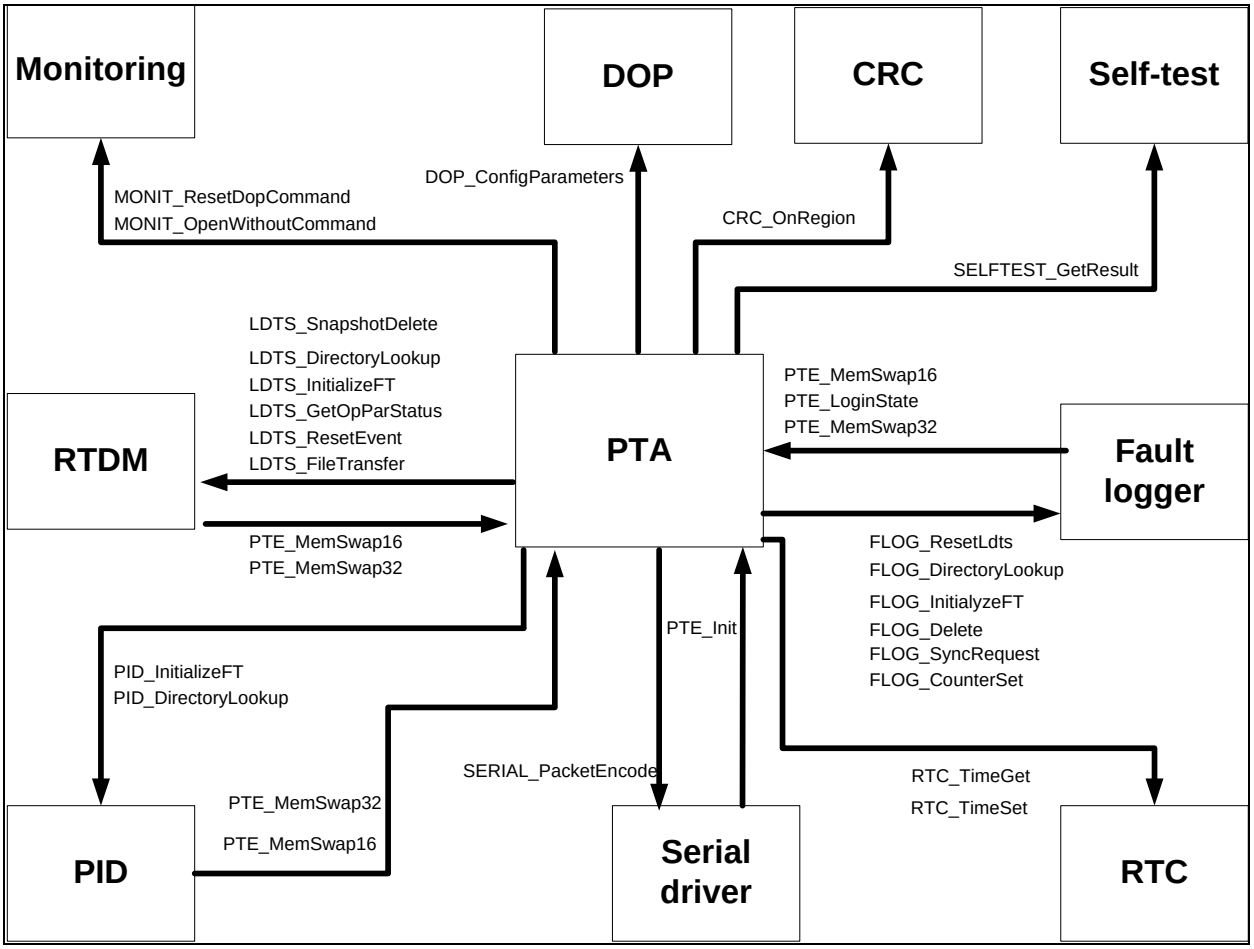

Figure 3-1: Inter-Module Dependencies Diagram

3.2.1.1 PTA Protocol Module

Figure 3-1 shows the Inter-Module Dependencies Diagram of the PTA Protocol Module.

3.2.2 Inter-Process Dependencies

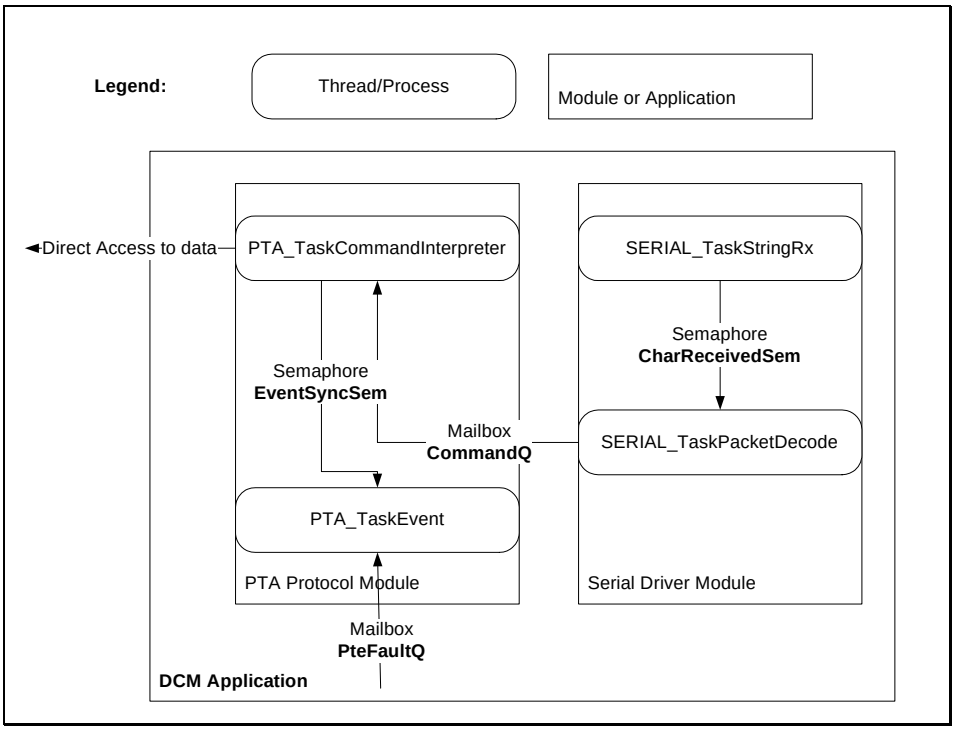

Figure 3-2: Inter-Process Dependencies Diagram

3.2.3 Data Dependencies

Not applicable.

3.3 Interface Description

This section describes the functions of the PTA Protocol Module. The function name, a description of the function, input and output parameters required by the function.

3.3.1 PTA Protocol Module functions

The Portable Test Application Module implements the following functions (sorted alphabetically):

3.3.1.1 PTA Protocol External Interface

Function Name	void PTE_Init (void)
Description	Creates all tasks related to the PTA and initializes variables
Input	Priority of the first created task
Output	None
Return	None

Function Name	Bit PTE_LoginState (void)
Description	This function returns the status of the PTA login state.
Input	None
Output	TRUE = The PTA is logged in FALSE = The PTA is not logged in
Return	None

Function Name	Unsigned int PTE_MemSwap16 (unsigned int uiValue)
Description	This function swaps the order of byte into a 16 bits value to switch between little endian and big endian and vice versa.
Input	Value to swap
Output	Value swapped
Return	None

Function Name	Unsigned long PTE_MemSwap32 (unsigned long ulValue)
Description	This function swaps the order of byte into a 32 bits value to switch between little endian and big endian and vice versa.
Input	Value to swap
Output	Value swapped
Return	None

3.3.1.2 PTA Protocol Internal Interface

Function Name	Void PTE_DateConv2Hex (unsigned int *uiYear, unsigned char *ucMonth, unsigned char *ucDay, INFO_KeilDate *ucStr)
Description	This function converts the string of the date and place converted values into provided pointer
Input	Date Pointer
Output	Date
Return	None

Function Name	Void PTE_DCUCodeStatusReq (PTE_NEU_CmdStruct *pMessage)
Description	This function receives Code Status request from PTA
Input	Pointers to the received message
Output	None
Return	None

Function Name	Void PTE_DCUConfigReq (PTE_NEU_CmdStruct *pMessage)
Description	This function receives configuration from PTA
Input	Pointers to the received message
Output	None
Return	None

Function Name	Void PTE_DCUToPteError (unsigned char ucStatus, unsigned char ucPTESeq)
Description	This function sends error message to PTA
Input	Communication status
Output	None
Return	None

Function Name	Void PTE_EventDelete (PTE_NEU_CmdStruct *pMessage)
Description	This function deletes event log or snapshot file
Input	Pointers to the received message
Output	None
Return	None

Function Name	Void PTE_FileStat (PTE_NEU_CmdStruct *pMessage)
Description	This function executes the PTE_FileStat command in answer to the PTE_FileReq question from PTA.
Input	Pointers to the received message
Output	None
Return	None

Function Name	Void PTE_FileXferOverIn (PTE_NEU_CmdStruct *pMessage)
Description	This function receives the FileXferOver message
Input	Pointers to the received message
Output	None
Return	None

Function Name	Void PTE_FileXferOverOut (unsigned char ucStatus)
Description	This function sends the FileXferOver message
Input	Origin of the request Sequence number
Output	None
Return	None

Function Name	Void PTE_Graph_Options_Req (PTE_NEU_CmdStruct *pMessage)
Description	This function receives Graph Options request from PTE
Input	Pointers to the received message
Output	None
Return	None

Function Name	Void PTE_LastPackIn (PTE_NEU_CmdStruct *pMessage)
Description	This function receives the last packet
Input	Pointers to the received message
Output	None
Return	None

Function Name	Void PTE_LastPackOut (unsigned char ucOrg, unsigned char ucPTE_Seq)
Description	This function sends the last packet
Input	Origin of the request Sequence number
Output	None
Return	None

Function Name	Void PTE_LocationId (unsigned char ucOrg, unsigned char ucPTE_Seq)
Description	This function sends the location ID
Input	Origin of the request Sequence number
Output	None
Return	None

Function Name	Void PTE_Login (unsigned char ucStatus, PTE_NEU_CmdStruct *message)
Description	This function login to the PTA
Input	Communication status Login message
Output	None
Return	None

Function Name	Void PTE_LoginReq (PTE_NEU_CmdStruct *pMessage)
Description	This function receives login request from PTE
Input	Origin of the request Sequence number
Output	None
Return	None

Function Name	Bit PTE_LoginState (Void)
Description	This function is used to manage RTDM sessions. It sends data automatically to the PTA.
Input	Sequence number
Output	None
Return	None

Function Name	Void PTE_Logout (unsigned char ucStatus, unsigned char ucPTE_Seq)
Description	This function executes the PTA logout command
Input	Communication status Sequence number received from the PTA, 0 if automatic logout
Output	None
Return	None

Function Name	Void PTE_LogoutReq (PTE_NEU_CmdStruct *pMessage)
Description	This function executes the command logout depending of the type of the data received
Input	Pointer on the PTA command structure
Output	None
Return	None

Function Name	Void PTE_NextPackIn (PTE_NEU_CmdStruct *pMessage)
Description	This function receives the next packet
Input	Pointers to the received message
Output	None
Return	None

Function Name	Void PTE_NextPackOut (unsigned char ucOrg, unsigned char ucPTE_Seq)
Description	This function send the next packet
Input	Origin of the request Sequence number
Output	None
Return	None

Function Name	Void PTE_NextPackReqIn (unsigned char ucOrg, unsigned char ucPTE_Seq)
Description	This function asks for the next packet
Input	The origin of the command (Car Network or PTA)
Output	None
Return	None

Function Name	Void PTE_NextPackReqOut (void)
Description	This function is used only when the PTA upload an DCU Application code into the DCU.
Input	None
Output	None
Return	None

Function Name	Void PTE_SciVersion (unsigned char ucOrg, unsigned char ucIndex, unsigned char ucPTE_Seq)
Description	This function send the requested SCI versions
Input	Origin of the request Index of the SciVersion requested Sequence number
Output	None
Return	None

Function Name	Void PTE_SciVersionPack (unsigned char ucIndex, PTE_SystemSciVersionStruct *pSciVersion, unsigned char ucPTESeq)
Description	This function package the SCI information into the PTE_SystemSciVersionStruct structure format ready to sends on the PTA or on the CommunicationGateway port
Input	Index of the requested SciVersion Structure to fill with version info Sequence number
Output	None
Return	None

Function Name	Void PTE_SelfTestReq (PTE_NEU_CmdStruct *pMessage)
Description	This function executes requested selftest and sends the result
Input	Pointers to the received message
Output	None
Return	None

Function Name	Void PTE_SystemCountersResetReq (PTE_NEU_CmdStruct *pMessage)
Description	This function resets the system counters
Input	Pointers to the received message: 1 Door Cycles 2 Operating Hours 3 Event Occurrence
Output	None
Return	None

Function Name	Void PTE_TaskCommandInterpreter (void)
Description	This function verify if the PTA is logged in before executing the received command
Input	None
Output	None
Return	None

Function Name	Void PTE_TaskEvent (Void)
Description	This Task is used to Send events on the Network (and the PTA) when they occurs.
Input	None
Output	None
Return	None

Function Name	Void PTE_TimeSet (PTE_NEU_CmdStruct *pMessage)
Description	This function sets the time
Input	Pointers to the received message
Output	None
Return	None

Function Name	Void PTE_TimeStatus (unsigned char ucPTE_Seq)
Description	This function sets the time
Input	Pointers to the received message
Output	None
Return	None

Function Name	Void PTE_TimeSend (PTE_NEU_CmdStruct *pMessage)
Description	This function sends time to PTE
Input	Sequence number
Output	None
Return	None

Function Name	Void PTE_Version (unsigned char ucOrg, unsigned char ucPTE_Seq)
Description	This function sends the description files versions
Input	Origin of the version request
Output	None
Return	None

Function Name	Void PTE_VersionReq (PTE_NEU_CmdStruct *pMessage)
Description	This function receives the version request from the PTA
Input	Pointers to the received message
Output	None
Return	None

3.4 Detailed Design

3.4.1 Portable Test Application Module Design

3.4.1.1.1 PTE_Init

The PTA module is started with a call to the function PTE_Init. This function starts 2 tasks used by the PTE: PTE_TaskCommandInterpreter, and PTE_TaskEvent.

3.4.1.1.2 PTE_DCUCodeStatusReq

The Master end sends the PTE_DCUCodeStatusReq to the DCU Application. The parameters received with this command are CODE_STAT_BOOT for the Boot Loader, CODE_STAT_APPLICATION for the DCU Application and CODE_STAT_CAR NETWORK for the Car Network. The answer to this request is the size and CRC of the specified application requested. The answer is returned to the Master end using the PTE_DCUCodeStatus command.
The information regarding the code status of the DCU Application and DCU Boot loader are found in Flash memory.
The format is described in #201111489 Portable Test Application -- Interface Control Document (ICD).

3.4.1.1.3 PTE_DCUCodeStatus

The DCU Application sends the command PTE_DCUCodeStatus to the Master end in answer to the PTE_DCUCodeStatusReq.
The fields filled in the structure follow the format described in #201111489 Portable Test Application -- Interface Control Document (ICD).

3.4.1.1.4 PTE_DCUConfig

The PTE_DCUConfig is sent by the Master end both to send configuration and to retrieve configuration of the door operator in the DCU Application. The parameters are saved or retrieved by the function DOP_ConfigParameters. The answer from the DCU Application to this command is PTE_DCUConfigStatus. The DCU Application validates the received configuration parameters to ensure they are within the valid range. A validation is also performed in the Master end.
The format is described in #201111489 Portable Test Application -- Interface Control Document (ICD).

3.4.1.1.5 PTE_DCUConfigStatus

The DCU Application sends PTE_DCUConfigStatus command to the Master end in answer to the PTE_DCUConfig.
The format is described in #201111489 Portable Test Application -- Interface Control Document (ICD).

3.4.1.1.6 PTE_DCUtoPteError

The DCU Application sends this command when a command received by the DCU from the Master end is not recognized or not supported due to the command itself or its data.
The format is described in #201111489 Portable Test Application -- Interface Control Document (ICD).

3.4.1.1.7 PTE_FileStat

The command PTE_FileStat is used to answer to the PTE_FileReq sent by the Master end. The DCU Application sends this command to the Master end.
The format is described in #201111489 Portable Test Application -- Interface Control Document (ICD). Figure 3-3 describes the process of the DCU Application for this command.

3.4.1.1.8 PTE_FileReq

The Command PTE_FileReq is used to perform a file transfer between the Master end and the DCU Application. This command is sent by the PTA to request different types of files. The PTA Protocol Module answer is PTE_FileStat command.
Table 3-1 shows the File index number and file type that can be transferred from the DCU Application to the Master end. The file index 0 is not valid and is reserved for the DCU Boot loader to upgrade the DCU Application. When the command PTE_FileReq is sent with the file index 0, the PTA Protocol Module sends PTE_FileStat command with FS_ACCESS_UNAVAIL parameter to pass back to the Master end.
If the file requested is the Internal Event Log File, the Event named “Event Log Retrieved by Master end” is sent to Fault Logger.
The format is described in #201111489 Portable Test Application -- Interface Control Document (ICD).

Table 3-1: PTA File Type Implemented in DCU Application

File Index	File type	Description	Note
0	2	DCU Application code transferred to the DCU Boot loader	
1	1	RAM Memory dump (SRAM and NVRAM)	
2	257	Event Description File	
3	258	Internal Event Log File	
4	259	Operating Parameters Definition File	
5	260	Network Log File	Used only for deletion, this file cannot be retrieved by PTA
6	261	Snapshot File	Snapshot file #1
7	261	Snapshot File	Snapshot file #2
8	261	Snapshot File	Snapshot file #3
9	261	Snapshot File	Snapshot file #4
10	261	Snapshot File	Snapshot file #5
11	261	Snapshot File	Snapshot file #6
12	261	Snapshot File	Snapshot file #7
13	261	Snapshot File	Snapshot file #8
14	261	Snapshot File	Snapshot file #9
15	261	Snapshot File	Snapshot file #10
16	261	Snapshot File	Snapshot file #11
17	261	Snapshot File	Snapshot file #12
18	261	Snapshot File	Snapshot file #13
19	261	Snapshot File	Snapshot file #14
20	261	Snapshot File	Snapshot file #15
21	261	Snapshot File	Snapshot file #16
22	261	Snapshot File	Snapshot file #17
23	261	Snapshot File	Snapshot file #18
24	261	Snapshot File	Snapshot file #19
25	261	Snapshot File	Snapshot file #20
26	262	Door Movement Information	Opening Movement
27	262	Door Movement Information	Closing Movement
28	262	System Counters File	System Counters structure

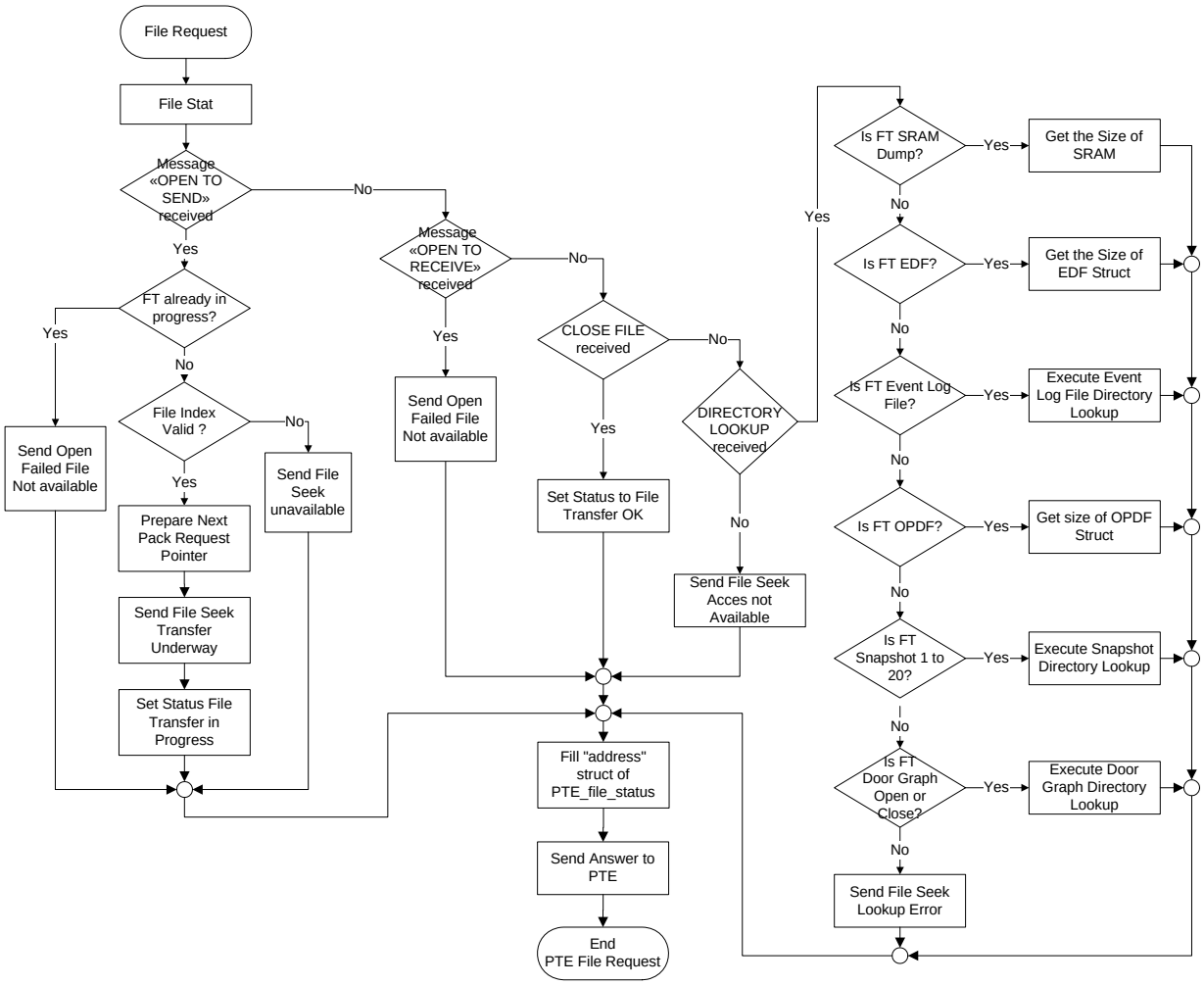

Figure 3-3: PTA File Request and File Stat Flowchart

3.4.1.1.9 PTE_FileDeleteReq

The PTE_FileDeleteReq is sent by the Master end to delete the event log (Car Network log) or a specified Snapshot. FT_EVENT_LOG_FILE as parameter in the PTE_FileDeleteReq message indicates that the function has to delete the complete Event Log File. In this case, Event named “Event Log Erased by Master end” is logged right after all faults are deleted. FT_LDTS_FILE_x as parameter indicates that the function has to delete the Snapshot number x. FT_NETWORK_LOG_FILE indicates that the function has to delete the Car Network Events Log. Setting a bit in the DCU Synchronization Message indicates to the Car Network to delete its log.
The answer from the DCU Application to the PTE_FileDeleteReq command is the PTE_FileDeleteStatus that confirms the request file is deleted.
The format is described in #201111489 Portable Test Application -- Interface Control Document (ICD).

3.4.1.1.10 PTE_FileDeleteStatus

The DCU Application sends the command PTE_FileDeleteStatus to the Master end in answer to the PTE_FileDeleteReq.
The format is described in #201111489 Portable Test Application -- Interface Control Document (ICD).

3.4.1.1.11 PTE_FileXferOver

This command is used to confirm the status of the file transfer. The sender is the Master end. This command is sent in answer to the DCU Application PTE_LastPack.
The format is described in #201111489 Portable Test Application -- Interface Control Document (ICD).

3.4.1.1.12 PTE_LastPack

This command is used to send the last packet of data in the file transfer process. The sender is the DCU Application. This command is sent in answer to the Master end request PTE_NextPackReq.
The format is described in #201111489 Portable Test Application -- Interface Control Document (ICD).

3.4.1.1.13 PTE_Location_IDReq

The PTE_Location_IDReq is sent by the Master end to request the Location ID of the DCU in the car and the car number. The answer from the DCU Application to this command is PTE_Location_ID with the location identification of the DCU (project's specific) and the car number.
The format is described in #201111489 Portable Test Application -- Interface Control Document (ICD).

3.4.1.1.14 PTE_Location_ID

The DCU Application sends the command PTE_Location_ID to the Master end in answer to the PTE_Location_IDReq.
The format is described in #201111489 Portable Test Application -- Interface Control Document (ICD).

3.4.1.1.15 PTE_Login

The PTE_Login is sent by the PTA Protocol Module to confirm the success of the login request. The DCU Application sends a status code (see Table 3-2: Status Code for PTA Login and PTA Logout command) depending of the result. Status code 2 is reserved for the indication of the DCU Boot Loader login answer.
The format is described in #201111489 Portable Test Application -- Interface Control Document (ICD).

Table 3-2: Status Code for PTA Login and PTA Logout command

Number	Status Code	Description
0	PTE_LOG_NOTLOGGED	Indicate that the PTA is not logged to the target
1	PTE_LOG_ERROR	Indicate that an error occurred during login
2	PTE_LOG_DCUBOOTLOADER	Indicate that the PTA is logged to the target Boot Loader
3	PTE_LOG_DCUAPPLICATION	Indicate that the PTA is logged to the target Application
4	PTE_LOG_REBOOT_DCU_APP	Target reboot is in progress

3.4.1.1.16 PTE_LoginReq

The PTA sends this command to log into the target application. The login is valid until the PTE_LogoutReq is received by the target application, the cable is disconnected, or no valid command is received within 60 seconds. The DCU Application sends the PTE_Login answer.
The format is described in #201111489 Portable Test Application -- Interface Control Document (ICD).

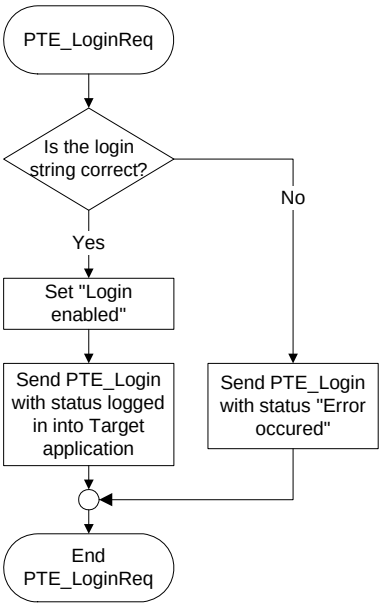

Figure 3-4: PTE_LoginReq

3.4.1.1.17 PTE_Logout

The PTE_Logout is sent by the DCU Application to confirm the success of the logout request. The DCU Application sends a status code (see Table 3-2: Status Code for PTA Login and PTA Logout command) depending on the result. Status code 2 is reserved for the indication of the DCU Boot loader logout answer.
The DCU Application sends this command with the status codes 0 if the Master end was not logged in before sending other commands.
The format is described in #201111489 Portable Test Application -- Interface Control Document (ICD).

3.4.1.1.18 PTE_LogoutReq

The Master end sends this command to logout from the DCU Application. The DCU Application sends the PTE_Logout answer.
The format is described in #201111489 Portable Test Application -- Interface Control Document (ICD).

3.4.1.1.19 PTE_NextPack

This command is used to send data in the file transfer process. The sender is the DCU Application. This command is sent in answer to the Master end request PTE_NextPackReq.
The format is described in #201111489 Portable Test Application -- Interface Control Document (ICD).

3.4.1.1.20 PTE_NextPackReq

This command is used to request a packet in the file transfer protocol. The sender is the Master end. The DCU Application sends a PTE_NextPack or PTE_LastPack command that contains the data to be transferred to answer that request.
The format is described in #201111489 Portable Test Application -- Interface Control Document (ICD).

3.4.1.1.21 PTE_SciVersion

The command PTE_SciVersion is used to answer the PTE_VersionRq sent by the Master end. The DCU Application sends this command to the Master end.
The fields filled in the structure follow the format described in #201111489 Portable Test Application -- Interface Control Document (ICD).

The index number of SCI is project's specific. The information regarding the version of the DCU Application, Boot Loader Application, and Operating System are found in the Flash memory. The version information for the Car Network Gateway Application is retrieved from the Car Network Synchronization Message.

3.4.1.1.22 PTE_SelfTestReq

The PTE_SelfTestReq is sent by the Master end to request self-test results from DCU Application.
The format is described in #201111489 Portable Test Application -- Interface Control Document (ICD).

3.4.1.1.23 PTE_SelfTestStatus

The DCU Application sends the command PTE_SelfTestStatus to the Master end in answer to the PTE_SelfTestReq.
The format is described in #201111489 Portable Test Application -- Interface Control Document (ICD).

3.4.1.1.24 PTE_SystemCounterResetReq

The PTE_ SystemCounterResetReq is sent by the Master end to request a reset of the system counters.
The format is described in #201111489 Portable Test Application -- Interface Control Document (ICD).

3.4.1.1.25 PTE_TaskCommandInterpreter

The PTE_TaskCommandInterpreter is used to receive and execute messages from the Serial Driver Module. Table 3-3 shows the complete list of messages supported and implemented in the DCU Application.

Table 3-3: PTA Messages implemented in DCU Application

Message #	Message Name	Comments
3	PTE_FileReq	Received from PTA Application
4	PTE_FileStat	Sent to the PTA Application
7	PTE_EventSyncRq	Received from PTA Application
8	PTE_Event	Sent to the PTA Application
13	PTE_DataRq	Received from PTA Application
14	PTE_Data	Sent to the PTA Application
15	PTE_VersionRq	Received from PTA Application
16	PTE_Version	Sent to the PTA Application
17	PTE_SciVersion	Sent to the PTA Application
32	PTE_NextPackReq	Received from PTA Application
33	PTE_NextPack	Sent to the PTA Application
34	PTE_LastPack	Sent to the PTA Application
35	PTE_FileXferOver	Received from PTA Application
48	PTE_Location_IDReq	Received from PTA Application
49	PTE_Location_ID	Sent to the PTA Application
50	PTE_DCUtoPteError	Sent to the PTA Application
64	PTE_TimeReq	Received from PTA Application
65	PTE_TimeStatus	Sent to the PTA Application
66	PTE_SelfTestReq	Received from PTA Application
67	PTE_SelfTestStatus	Sent to the PTA Application
69	PTE_FileDeleteReq	Received from PTA Application
70	PTE_LoginReq	Received from PTA Application
71	PTE_Login	Sent to the PTA Application
72	PTE_LogoutReq	Received from PTA Application
73	PTE_Logout	Sent to the PTA Application
74	PTE_DCUCodeStatusReq	Received from PTA Application
75	PTE_DCUCodeStatus	Sent to the PTA Application
76	PTE_DCUConfig	Received from PTA Application
77	PTE_DCUConfigStatus	Sent to the PTA Application
78	PTE_FileDeleteStatus	Sent to the PTA Application

Message #	Message Name	Comments
79	PTE_ComTestReq	Received from PTA Application
80	PTE_ComTestStatus	Sent to the PTA Application
81	PTE_DoorCmdReq	Received from PTA Application
82	PTE_DoorCmdStatus	Sent to the PTA Application
83	PTE_SystemCounterResetReq	Received from PTA Application
84	PTE_SystemCounterResetStatus	Sent to the PTA Application

The DCU Application using the PTA Protocol Module can execute a command only if the Master end is logged in. Figure 3-5 shows the flowchart of the module. The Master end should send a Login command prior to any other command. If the Master end is not logged in and it sends a valid command, the DCU Application answers the PTE_Logout message with a status that the Master end is not logged in. Otherwise, if the command is valid, the command is executed and the appropriate message is sent to the PTA Application.
If the CRC of the received packet does not match the calculated CRC, the packet is discarded. If the packet received contains a valid CRC but a command that is not supported, the DCU calls the PTE_DCUToPteError function indicating that the command is not supported.

The commands implemented in the DCU Application are described in the following sections. The implementation of the commands follows document #201111489 Portable Test Application -- Interface Control Document (ICD).

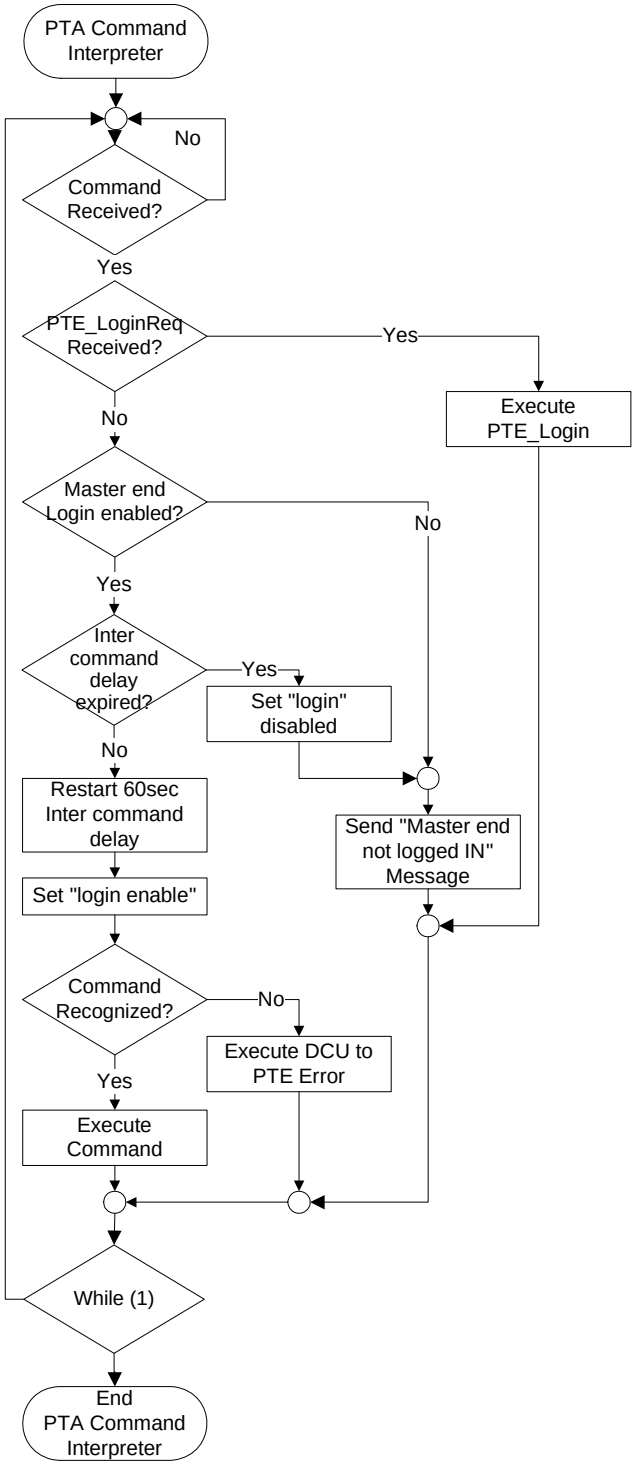

Figure 3-5: PTA Command Interpreter Flowchart

3.4.1.1.26 PTE_EventSyncRq

When PTE_EventSyncRq is received from Master end, all events in the PTA Event queue are removed and the queue is filled with active events.

3.4.1.1.27 PTE_Event

The PTE_Event is sent to the Master end whenever a new fault occurs while the Master end is connected. A task, the PTE_TaskEvent, is used to monitor and send faults to the Master end if this one is connected and requests it. Otherwise, the fault is simply not sent to the Master end.

3.4.1.1.28 PTE_TimeReq

The PTE_TimeReq is sent by the Master end to request the time that the DCU Application uses. The answer from the DCU Application to this command is PTE_TimeStatus.
The format is described in #201111489 Portable Test Application -- Interface Control Document (ICD).

3.4.1.1.29 PTE_TimeStatus

The DCU Application sends the command PTE_TimeStatus to the Master end in answer to the PTE_TimeReq.
The format is described in #201111489 Portable Test Application -- Interface Control Document (ICD).

3.4.1.1.30 PTE_Version

The command PTE_Version is used to answer the PTE_VersionRq sent by the Master end. The DCU Application sends this command to the Master end. The fields filled in the structure follow the format described in #201111489 Portable Test Application -- Interface Control Document (ICD).

3.4.1.1.31 PTE_VersionRq

The PTE_VersionRq is sent by the Master end to request the Software versions. The answer from the DCU Application is PTE_SciVersion or PTE_Version depending of the parameter received in the PTE_VersionRq. If the parameter received is VERSION_REQUEST, the answer is PTE_Version with the versions of the EDF and OPDF. If the parameter received is SCI_VERSION_REQUEST, the answer is PTE_Sci_Version with the version of the SCI requested. If the SCI requested is index 0, then all SCI are sent to the Master end. This PTA Variable contains collection of SCI versions (Maximum 10 SCI versions) depending on PTE_VersionRq request type
If the parameter received in the PTE_VersionRq variable is SCI_LAST_VERSION_REQUEST, the answer is PTE_Sci_Version with the version of the last SCI. Index and name of each SCI is project's specific.
The parameters and the formats are described in #201111489 Portable Test Application -- Interface Control Document (ICD).

3.4.1.1.32 PTE_DataRq

The PTE_DataRq is sent by the Master end to request the status of the DCU operating parameters. The answer from the DCU Application is PTE_Data.
The format is described in #201111489 Portable Test Application -- Interface Control Document (ICD).

3.4.1.1.33 PTE_Data

The PTE_Data is sent by the DCU Application and contains the status of all operating parameters. The DCU Application sends this command to the Master end. The format of the structure transferred is described in #201111489 Portable Test Application -- Interface Control Document (ICD).

3.4.1.1.34 PTE_SystemCounterResetStatus

The DCU Application sends PTE_ SystemCounterResetStatus command to the Master end in answer to the PTE_ SystemCounterResetReq.
The format is described in #201111489 Portable Test Application -- Interface Control Document (ICD).

3.4.2 Data Entities Detailed Design

Not applicable.